

Day 9 — Sigma Rule Structure, Writing & Testing

Name: Mohammed Farhan

Role: Associate – Information Security (Intern)

Organization: CyberShelter UAE

Date: 8 July 2026

Objective

To understand Sigma's role as a vendor-agnostic detection rule format, study its complete field structure, and write three original Sigma rules from scratch — each converted successfully into a real SIEM query to prove they work.

What Is Sigma

Sigma solves the same problem Day 6 first introduced with SPL vs KQL — every SIEM has its own query syntax, so a detection written for Splunk doesn't work in Elastic without a rewrite. Sigma provides one generic, human-readable YAML format that a converter (sigma-cli or Uncoder.IO) translates into any target SIEM's native query language, similar to how a PDF renders correctly across different printers regardless of brand. It was created and is maintained by SigmaHQ, a community-driven project that also publishes thousands of ready-made, peer-reviewed rules covering known attack techniques, which teams can adopt directly or use as a reference for writing their own. Because the format is open and platform-agnostic, it has become a de facto industry standard for sharing detections across organizations, regardless of which specific SIEM each one runs. This portability also future-proofs detection work — if a company migrates from Splunk to Elastic, existing Sigma rules can simply be re-converted rather than rewritten from scratch. For a detection engineer, this makes Sigma less about learning "one more tool" and more about learning a transferable skill that stays valuable across employers and technology stacks.

Sigma Rule Anatomy

Field	Purpose
title	Short, human-readable name of the detection
id	Unique UUID identifying the rule
status	Lifecycle stage (experimental/test/stable/deprecated)
description	Plain-English explanation of what and why
references	Links supporting the detection (MITRE ATT&CK, etc.)
author / date	Accountability and version tracking
logsource	Where the data comes from — product, category, service
detection	The actual matching logic — selection blocks + condition
falsepositives	Honest list of legitimate activity that could also trigger this rule
level	Severity — informational/low/medium/high/critical

Rules Written and Tested

Rule 1 — Windows Defender Real-Time Protection Disabled

title: Windows Defender Real-Time Protection Disabled

id: e8a23afb-f0b1-40e3-9704-1ca4f15e9452

status: test

description: Detects when Windows Defender real-time protection is disabled, which may indicate an attacker clearing the way to deploy malware undetected.

references:

- <https://attack.mitre.org/techniques/T1562/001/>

author: Mohammed Farhan

date: 2026/07/09

logsource:

product: windows

service: windows_defender

detection:

selection:

EventID: 5001

condition: selection

falsepositives:

- Installing another antivirus/security product

- Installing or running penetration testing tools such as Metasploit or FlareVM, which may be falsely flagged by Defender

level: high

UNCODER.IO
powered by SOC Prime, Inc.

Sigma

```
1 title: Windows Defender Real-Time Protection Disabled
2 id: e8a23afb-f0b1-40e3-9704-1ca4f15e9452
3 status: test
4 description: Detects when Windows Defender real-time protection is disabled, which
5               may indicate an attacker clearing the way to deploy malware undetected.
6 references:
7   - https://attack.mitre.org/techniques/T1562/001/
8 author: farhan
9 date: 2026/07/09
10 logsource:
11   product: windows
12   service: windows_defender
13 detection:
14   selection:
15     EventID: 5001
16   condition: selection
17 falsepositives:
18   - Installing another antivirus/security product
19   - Installing or running penetration testing tools such as Metasploit or FlareVM
20     , which may be falsely flagged by Defender
21 level: high
```

Converted SPL (Uncoder.IO):

- `source=WinEventLog:* AND EventID="5001"`

Uncoder AI Grow from rule translation to threat research with full priv

Splunk Query (SPL)

```
1 source=WinEventLog:* AND EventID="5001"
2
3 `` name: Windows Defender Real-Time Protection Disabled
4 `` uuid: e8a23afb-f0b1-40e3-9704-1ca4f15e9452 ``
5 `` author: farhan ``
6 `` licence: DRL 1.1 ``
7
8
9
```

Rule 2 — PowerShell Encoded Command Execution

title: PowerShell Encoded Command Execution

id: d3b5ab11-3904-4f35-a578-b1a7325613cb

status: test

description: Detects execution of PowerShell commands using the -EncodedCommand flag, commonly used by attackers to obfuscate malicious commands and evade text-based detection.

references:

- <https://attack.mitre.org/techniques/T1059/001/>

author: Mohammed Farhan

date: 2026/07/09

logsource:

product: windows

service: powershell

detection:

selection:

EventID: 4104

CommandLine|contains: '-EncodedCommand'

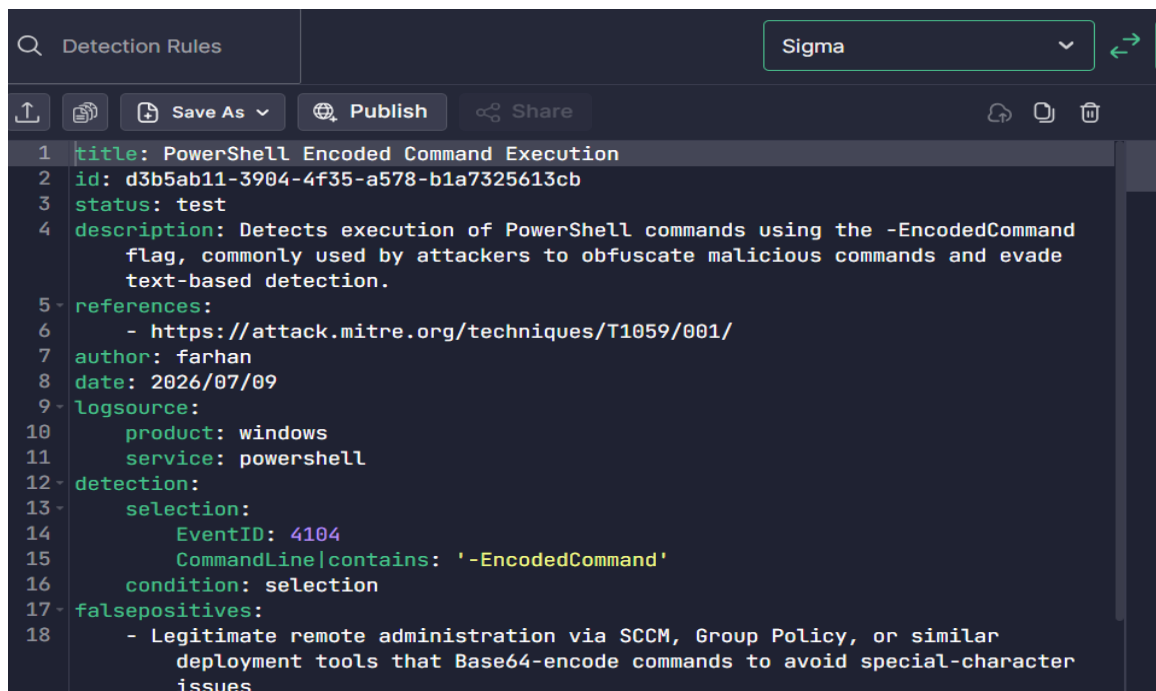
condition: selection

falsepositives:

- Legitimate remote administration via SCCM, Group Policy, or similar deployment tools that Base64-encode commands to avoid special-character issues

- Scheduled tasks or automation scripts that use encoded commands for reliability across systems

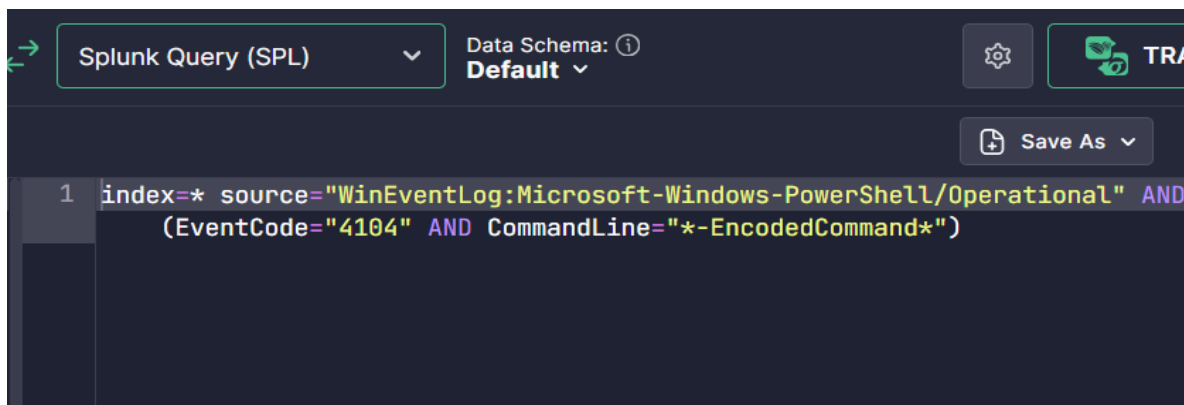
level: medium



```
1 title: PowerShell Encoded Command Execution
2 id: d3b5ab11-3904-4f35-a578-b1a7325613cb
3 status: test
4 description: Detects execution of PowerShell commands using the -EncodedCommand
               flag, commonly used by attackers to obfuscate malicious commands and evade
               text-based detection.
5 references:
6   - https://attack.mitre.org/techniques/T1059/001/
7 author: farhan
8 date: 2026/07/09
9 logsource:
10   product: windows
11   service: powershell
12 detection:
13   selection:
14     EventID: 4104
15     CommandLine|contains: '-EncodedCommand'
16   condition: selection
17 falsepositives:
18   - Legitimate remote administration via SCCM, Group Policy, or similar
     deployment tools that Base64-encode commands to avoid special-character
     issues
```

Converted SPL (Uncoder.IO):

- `index=* source="WinEventLog:Microsoft-Windows-PowerShell/Operational" AND (EventCode="4104" AND CommandLine="-EncodedCommand")`



```
1 index=* source="WinEventLog:Microsoft-Windows-PowerShell/Operational" AND
   (EventCode="4104" AND CommandLine="*-EncodedCommand*")
```

Technical note: Splunk's field name EventCode differs from Sigma's EventID — Uncoder.IO handled this translation automatically, illustrating exactly why a single Sigma rule can target multiple SIEMs without manual field-name adjustments.

Rule 3 — New Local Administrator Account Created

title: New Local Administrator Account Created

id: 7f4c2e91-8a6d-4b3f-9c5e-2d8a19f6b3e7

status: test

description: Detects a new user account being created or an existing account being added to the Administrators group. Each event is treated independently and does not confirm both occurred for the same account.

references:

- <https://attack.mitre.org/techniques/T1136/001/>
- <https://attack.mitre.org/techniques/T1098/>

author: Mohammed Farhan

date: 2026/07/09

logsource:

product: windows

service: security

detection:

selection1:

EventID: 4720

selection2:

EventID: 4732

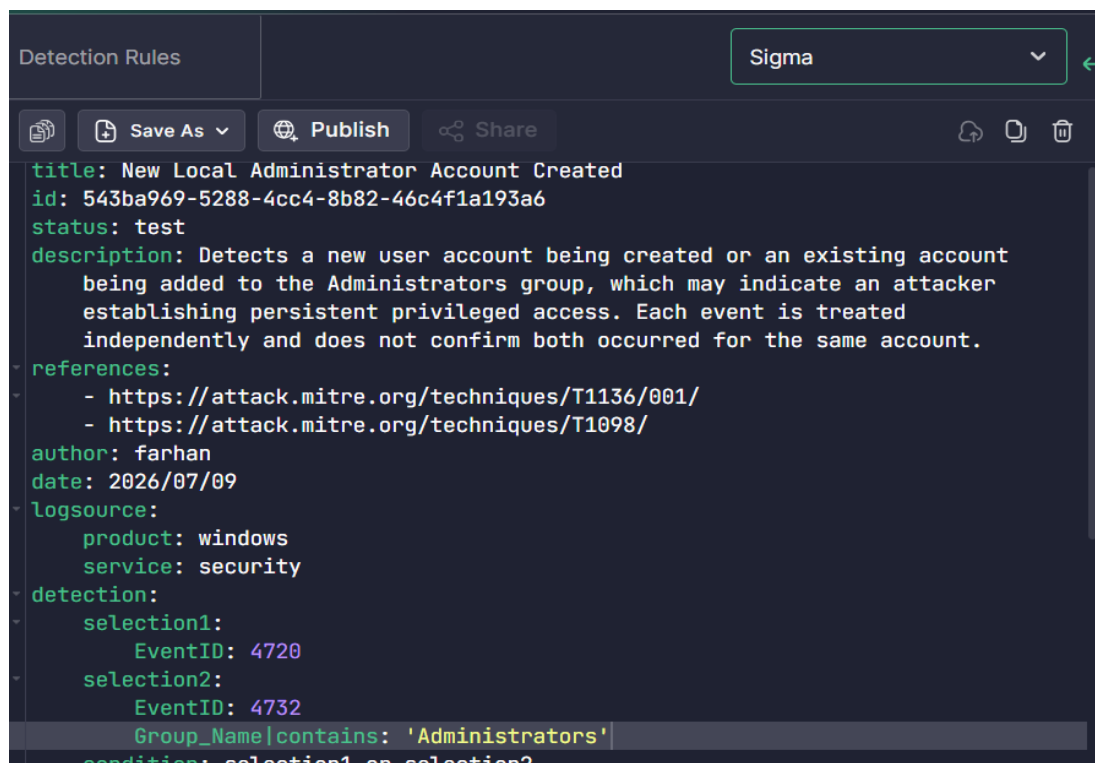
Group_Name|contains: 'Administrators'

condition: selection1 or selection2

falsepositives:

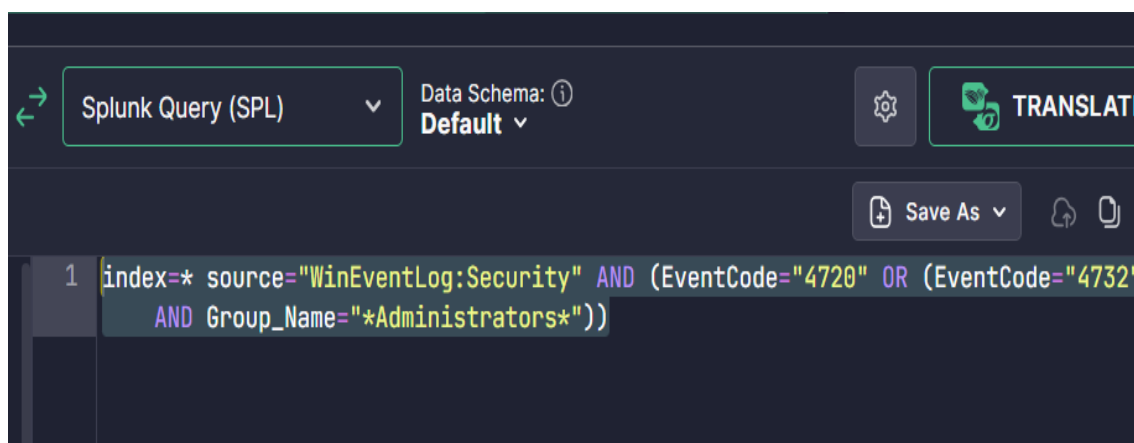
- New employee onboarding involving standard account creation
- Role change or promotion requiring elevated administrative access

level: medium



Converted SPL (Uncoder.IO):

- `index=* source="WinEventLog:Security" AND (EventCode="4720" OR (EventCode="4732" AND Group_Name="Administrators"))`



Deliverable

3 Sigma YAML files — rule1-defender-disabled.yml, rule2-powershell-encoded.yml, rule3-new-admin-account.yml — each containing all required fields, written from scratch, and successfully converted to valid Splunk SPL via Uncoder.IO. All three files are committed to the Git repo under week-3/day9/sigma-rules/.

Conclusion

Day 9 introduced Sigma as the industry-standard, vendor-agnostic detection format, directly extending the SPL-vs-KQL lesson from Day 6 into a single reusable rule syntax. Writing all three rules from scratch — rather than copying from SigmaHQ's repository — surfaced genuine syntax and logic errors (tab/space mismatches, a counterproductive exclusion filter, field-naming precision) that were debugged the same way Day 7's Wazuh decoder issues were resolved: by reading actual error output and testing incrementally rather than guessing. All three rules were verified as functionally correct by successful conversion to real Splunk queries.